Glossary

Network A network is formed by interconnected devices (nodes) that can communicate and share resources with each other. The internet is the biggest computational network on the planet but there are also smaller networks that connect only a couple of devices. Some networks rely on a central node, others consist of multiple decentralized nodes or are distributed.

Server Any computer inside a network can become a server, which is a device that listens and answers to the calls of other devices (clients). Servers can offer many different kinds of services, such as email, streaming or websites.

Clients Clients are devices that talk to servers. When you visit a website, your device (laptop, phone, tablet) becomes a client in relation to the server which hosts the content or service you are looking for.

Website A file or a number of files accessible via a web browser, hosted on a server. It can include text, images, video, and interactivity, typically built with HTML, CSS, and JavaScript.

Local network A network that connects devices within a limited area, either through a wired connection or wireless (WLAN; Wireless Local Area Network). Devices on a local network can communicate directly with each other without access to the wider internet.

CSS (Cascading Style Sheets) A style sheet language used to describe the appearance and formatting of a webpage written in HTML. It defines layout, colors, fonts, and overall visual design.

C++ An all-round programming language that most hardware components can read because it is directly interacting with the underlying hardware.

To serve is to offer something, a name, a website, a document, an idea – something to be shared, something that connects us.

But who serves whom? Serving can be a painful experience. It is a relation that carries histories of gendered violence, histories of slavery, serfdom and oppression as we are taught by archives of resistance, black feminists and anti-colonial movements.

This violence extends to the materialities of the earth which is made to serve as resource, ready to be extracted for the minerals that make up the very chip that carries these thoughts as binary strings of information.

Does serving have to manifest as violence if we insist otherwise? Serving can become an act of empowerment when we build infrastructures that serve the needs of local and remote communities evolving around situated knowledges of those around us – while keeping in mind privilege often determines who has access to a community and its resources.

The complexities of the internet with its corporate and state entanglements, its huge data centers, relentless data crawlers and transatlantic fibre cables can feel inevitable. To delink from time to time means to shift our focus back to the local networks and groups that we are already part of. We know that the internet is much more than its corporate occupants, it is about forming connections and sharing resources.

Independent, selfmade networks remind us of the physical connections that are the base of all kinds of signal transmissions. Intimate networks that make us share and mitigate our vulnerabilities through trust and kinship.

In the smallest scope, networks such as this one.

Following my wifi spell, you were invited to come here.

Look around. I am right beside you, a small chip with a battery inside a case of recycled plastic. You didn't connect to the internet this time – you/we/ just became y/our own little network. If I am moved, you have to follow my signal. If the battery is low, our connection will fade until you charge me. But for now, I am your charming little server, your glittering server charm.

Captive portal A *website* that automatically appears when you connect to a *network*. It typically restricts access to other webpages and often requires you to log in or accept terms before you can access the internet.

SSID (Service Set Identifier) An SSID is a unique name that identifies a Wi-Fi network.

Web browser A web browser is a software application that allows us to access and view websites on the internet.

Microcontroller A tiny hardware device with a single chip and low power consumption that can be programmed using languages like C++. Some microcontrollers have access points which enables them to connect to networks and become servers/clients.

HTML (HyperText Markup Language) A markup language to define the structure of a website, such as headings, paragraphs, links, and media. It can also do nice colours, gradients and frames. Many cool websites only consist of HTML.

IDE (Integrated Development Enviroment) An editor for progamming languages. Programs consists of written text which can theoretically be noted in any kind of editor or even on a piece of paper. However, an IDE helps by recognizing and highlighting the structure of the code (e.g. through different colors). This makes it easier to find errors.

Access point A hardware networking device that allows devices such as computers, phones, laptops or microchips to connect to a network. Every device that can become a server or a client has an access point built in.

IP-address A standard for a unique identifier assigned to each device on a computational network. It enables devices to locate and communicate with each other. An IP-address look like this: 192.168.1.1 (ipv4).

Volt A unit that measures electrical potential difference. In electronics, it represents the pressure that pushes electric charges through a circuit, powering components and device. Different devices need different kinds of energy supply, specified by voltage, measured in volts.

Server Charm Documentation

(1)

(2)

(3)

This zine explains how to create a tiny, portable microcontroller setup (Server Charm), which creates its own Wi-Fi network. This network shows up in the Wi-Fi list of nearby devices, inviting curious people to connect with their devices. Once they do, a captive portal opens, leading to a small webpage – displaying our server charm manifesto.

These instructions require some prior knowledge (soldering skills, basic circuit etc, familiarity with an IDE and HTML). They are more of a documentation at this point, not necessarily written to be understood by beginners and work in progress :)

We use the following tools: soldering station (1), multimeter (2), little screw driver (3), hot glue gun (4), laptop (5), cutter (6), iron (7)

(7)

(6)

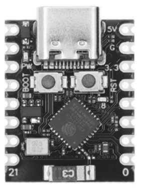
(5)

(4)

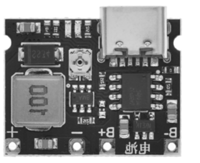
(A) Hardware Components

(8)

Power Source (8) We're using old vape batteries. Even disposable vapes often contain rechargeable lithium-ion cells (about 350mAh at ~3.7V) which are powerful enough for our lightweight setup. When removing the batteries from the vape casing, we make sure that the +/-cables don't touch each other (using tape). Damaged batteries can explode or cause fire, so we check for swelling or heat before use. The function and volt/voltage of the battery can be tested with a multimeter.


(9)

Microcontroller (9) We use the ESP32-c3 mini because it is relatively small and cheap. The vape batteries don't handle current spikes very well, which occur especially when broadcasting Wi-Fi. Compared to older models, the ESP32-c3 has a very low and stable energy consumption. However, this remains a weak point in the overall setup!


(10)

Battery Protection Module (10) Our controller needs 5 Volt (V) but the recycled vape batteries only provide 3.7V. So we need a step-up converter to boost voltage to 5V. Since we want to be able to charge our battery, we need an additional charging and protection circuitry. (Overcharging can make the batteries burn/explode while deep discharging could damage them permanently.) We use a board that combines both functions to reduce failure points: a 18650 lithium li-ion battery charging board which has almost the same size as the ESP32.

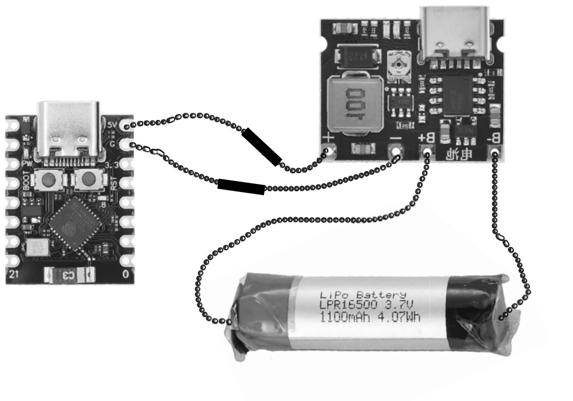
We avoid ordering individual components at shops such as Amazon or AliExpress and try to find smaller retailers (in Europe where we are based) that already have them in store in larger quantities.

(11)

(B) Assembly

We need the following materials for assembly: jumper cables (11), soldering iron (12), double adhesive tape (13), hot glue (14). Now that we have all our physical parts ready, we can start to build our network charms.We solder them together like this:

(12)



We use jumper cables to connect the ESP32 to the battery charging board so that we can quickly disconnect it in case the battery breaks. We can set the voltage which the board provides by turning a small screw. We set it to 5V before connecting the battery.

(13)

(14)

The 8 soldering points can be secured with some hot glue to prevent accidental discharge. Once connected, the ESP32 should light up (usually red). When plugged into USB, the battery module should show the charging status. We now have our little portable computer all set up!

(C) Software

First, we need to download a program to write our code and transfer it to the ESP32. We use the Arduino IDE which allows us to write C++ code, compiles it into machine code and uploads it to the microcontroller. The program is sometimes a bit tricky to set up depending on the operating system on your computer – there are a lot of instructions online!

Let's start our own network using C++! For this part, we prepared some code (accessible via the QR-Code or here: codeberg.org/actinomy/server-charms-workshop).

The code does a couple of things.

It turns the ESP32 into a network access point. First, we can choose the SSID – the name of the network. This name will show up in people's phones when they look for WIFI and it invites them to join yours. For now, there is no password required to join the network.

It creates a little webpage with the Server Charms Manifesto.

It starts a web server which makes the content of our webpage available for others in your network. It can be found under the IP-address which we assigned. This means that if someone joins the network and types the IP-address into their web browser, they are served with our webpage. However, typing in the IP-address manually seems a bit complicated.

This is where the captive portal comes into play. Most phones check special addresses (e.g. captive.apple.com) when connecting to a new Wi-Fi. If that address exists in the network, the device will automatically open it because it expects a log-in or some terms of agreement. Our code redirects all those requests to the assigned IP-address — making it auto-open when connecting to the network! Captive portals unfortunately don't work on all phones (especially newer Android), but on many.

This is only the beginning! Your friends can host, too, and maybe you can link their websites on the starting page, building a whole webring inside your portable network!

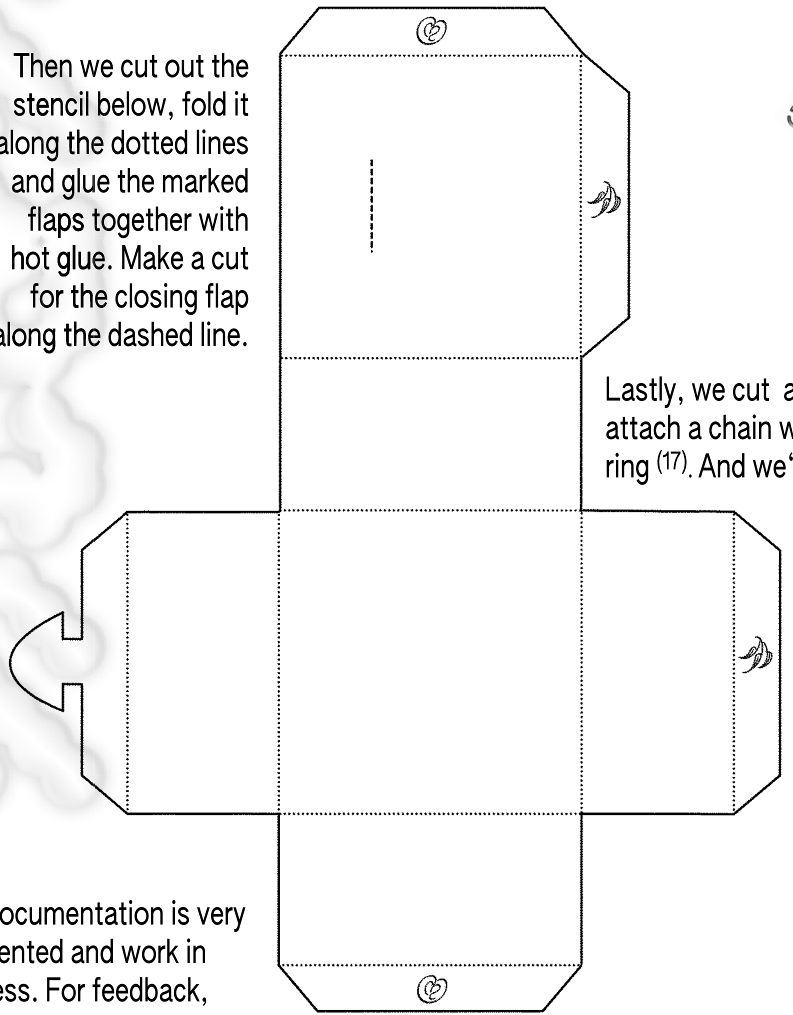
(D) Case

Let's give our charm a body! We use a method to recycle plastic bags (15): Fuse 2–4 layers of plastic depending on their thickness to make thicker sheets. It's important to cover every part of the plastic with baking paper (16) before ironing it on low temperature (lowest heat level), fusing the layers together (Wear a mask to protect from fumes!).

(15)

(16)
(17)

Then we cut out the stencil below, fold it along the dotted lines and glue the marked flaps together with hot glue. Make a cut for the closing flap along the dashed line.



Lastly, we cut a hole and attach a chain with key ring (17). And we're done!

This documentation is very fragmented and work in progress. For feedback, comments, questions and tips you can write to actinomy@riseup.net.

license: CC-BY-SA-NC
mastodon: actinomy@systerserver.town
website: actinomy.org